CareFlow: Clinical Pathway Process Mining - Project Report

Popuri Charan Sai · Data Analytics Intern, Infotact Solutions · Nov 8, 2026 · @Charan sai

Abstract

CareFlow shows where time is lost in an emergency-room patient journey by rebuilding each patient's real path from timestamped records. A Python generator simulates 10,000 patients and 68,440 events, with three problems planted on purpose. The data flows through Google BigQuery and dbt into PM4Py for process discovery and into Power BI for a manager's dashboard. The pipeline found all three problems without being told: 40.6% of X-Ray patients loop back to Triage, doctor waits average 48.5 minutes in busy hours against 17.8 otherwise, and the evening pharmacy averages 44.5 minutes against 18.2. A conformance check shows 90.35% of patients follow the ideal path, and the 965 who do not are exactly the loop-back patients. All data is simulated, so the results show that the method works, not facts about a real hospital.

1. Introduction and objectives

A hospital usually sees one average waiting time, and that number cannot show why patients wait. Process mining reads the timestamps in hospital records and rebuilds the path each patient actually took, so a manager can see which steps follow which and where time is lost. CareFlow applies this idea to an emergency-room pathway. It studies what already happened; it does not schedule patients.

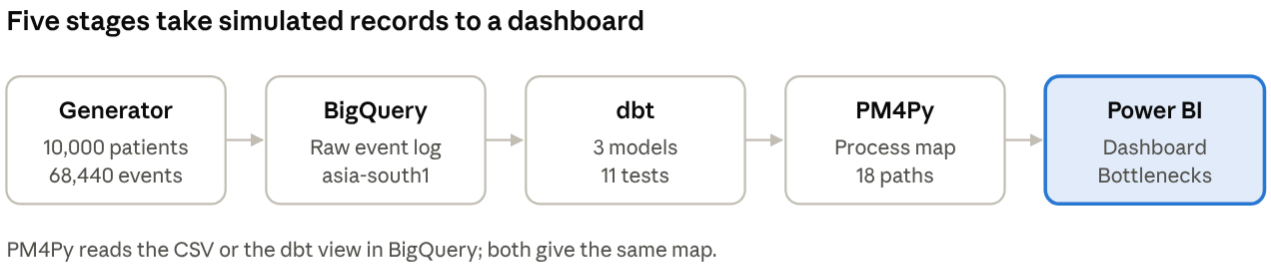
The project ran from 10 October to 10 November 2026 as an individual internship project at Infotact Solutions. Its objectives were:

1. Generate a realistic, reproducible event log with known problems planted in it.
2. Store the log in Google BigQuery and clean it with dbt models and tests.
3. Discover the real patient paths with PM4Py and check them against an ideal path.
4. Show the bottlenecks in a Power BI dashboard.
5. Keep all work in a public GitHub repository with daily commits.

No real hospital data was available, so the event log is simulated. Every finding in this report therefore shows that the pipeline works. None of them describes a real hospital.

2. System design

The pipeline has five stages: a Python generator, BigQuery, dbt, PM4Py and Power BI. PM4Py reads either the CSV file or the dbt view in BigQuery, and both give the identical process map.



pipeline · 5 stages

Event log format

Every row is one step a patient went through. The log has 10,000 patients, 68,440 events and 18 distinct patient paths over 30 days (about 333 patients a day).

Column	Meaning
Case_ID	One patient visit
Activity	The step that happened
Timestamp	When the step started
Condition	Fracture or injury, Chest pain, Stomach pain, Fever or infection, or Minor illness
Age_Group	Child, Adult or Senior
Ward	Ward name, filled only on the Ward Admission step

The steps are Registration, Triage, Doctor Consultation, X-Ray, Lab Test, Doctor Review, Ward Admission, Pharmacy, Billing and Discharge. Patients who are admitted skip Pharmacy and Billing. The generator uses a fixed random seed (42), so every run produces the same data.

The ideal path

The ideal path moves forward through numbered stages and never goes back or repeats a step. X-Ray and Lab Test share a stage, as do Ward Admission and Pharmacy.

Stage	Steps
0	Registration
1	Triage
2	Doctor Consultation
3	X-Ray, Lab Test
4	Doctor Review
5	Ward Admission, Pharmacy
6	Billing
7	Discharge

Planted problems

Three known problems are built into the simulated data, so process mining has something real to discover.

Problem	How it is built in
Loop-back	About 40% of X-Ray patients are sent back to Triage
Busy-hour doctor waits	20 to 60 extra minutes at 10:00 to 14:00 and 18:00 to 22:00, against 0 to 10 at other hours
Slow evening pharmacy	15 to 45 extra minutes at 18:00 to 23:00, against 0 to 5 at other times

3. Implementation

Each stage lives in its own folder of the public GitHub repository, so the work can be followed step by step.

Python event log generator

`data_generator/generate_events.py` creates each patient with a Condition and an Age_Group, then walks them through the steps with realistic gaps between them. It applies the 40% X-Ray to Triage loop-back, the busy-hour doctor waits and the slower

evening pharmacy. `validate_data.py` runs 9 quality checks on the output, and `check_waits.py` prints the planted waiting problems.

Google BigQuery

The CSV is loaded into the table `event_log_raw` in the dataset `careflow_raw`, in the asia-south1 region of the project careflow-ehr-04. The table definition is stored in `bigquery/schema.json`, with Ward as the only nullable column, and the setup steps are in `docs/bigquery_setup.md`.

dbt models

dbt reads the raw table and builds three views in the dataset `careflow_clean`:

Model	What it gives
event_log	The event log in the standard format, with an Event_Order column per patient
transition_times	Average minutes between every pair of consecutive steps
case_conformance	One row per patient: does the real path follow the ideal path?

The models are tested with 11 dbt tests.

PM4Py process discovery

`process_mining/discover_process.py` reads the event log from the CSV or from the BigQuery view, builds the directly-follows graph with PM4Py and saves the process map as an image. The graph shows 19 connections between steps.

Power BI dashboard

Power BI Desktop imports the dbt views from BigQuery. The file `powerbi/careflow_dashboard.pbix` has a bottleneck chart, KPI cards and a process-map page, plus a conformance page with two percentage cards, a bar chart by condition and slicers for Condition and Age_Group.

Version control

All work was committed directly to the main branch of a public repository, with one real change per day. Notebooks, the virtual environment, `profiles.yml` and the dbt `target/` and `logs/` folders are kept out of the repository on purpose.

4. Testing and data quality

The data was checked at three levels before any finding was drawn from it, and all checks pass.

Level	What was tested	Result
Python	`validate_data.py` runs 9 checks on the CSV	9 of 9 pass
dbt	11 tests on the BigQuery models	All pass
Cross-tool	dbt `case_conformance` against PM4Py	Both count 965 loop-back patients

The 9 Python checks are: no missing values; every patient starts with Registration and ends with Discharge; timestamps never go backwards; Ward appears only on Ward Admission rows; every Ward Admission row has a Ward; admitted patients skip Pharmacy and Billing; activity names are valid; Condition and Age_Group stay constant per patient; and there are no duplicate rows.

The process map PM4Py draws from the CSV and the one it draws from the BigQuery view are the same image, byte for byte, which shows the pipeline did not change the data on the way through. The measured loop-back rate is 965 of 2,377 X-Ray patients (40.6%), close to the 40% that was planted.

5. Results

The pipeline found all three planted problems without being told where to look. It also showed which patients leave the ideal path.

Process map

PM4Py discovered 18 different patient paths and 19 connections between steps. The arrow from X-Ray back to Triage is the planted loop-back: 965 of 2,377 X-Ray patients (40.6%) were sent back. The five most common paths cover about 73% of patients, and

the most common one (Registration, Triage, Doctor Consultation, Pharmacy, Billing, Discharge) is followed by 2,400 patients.

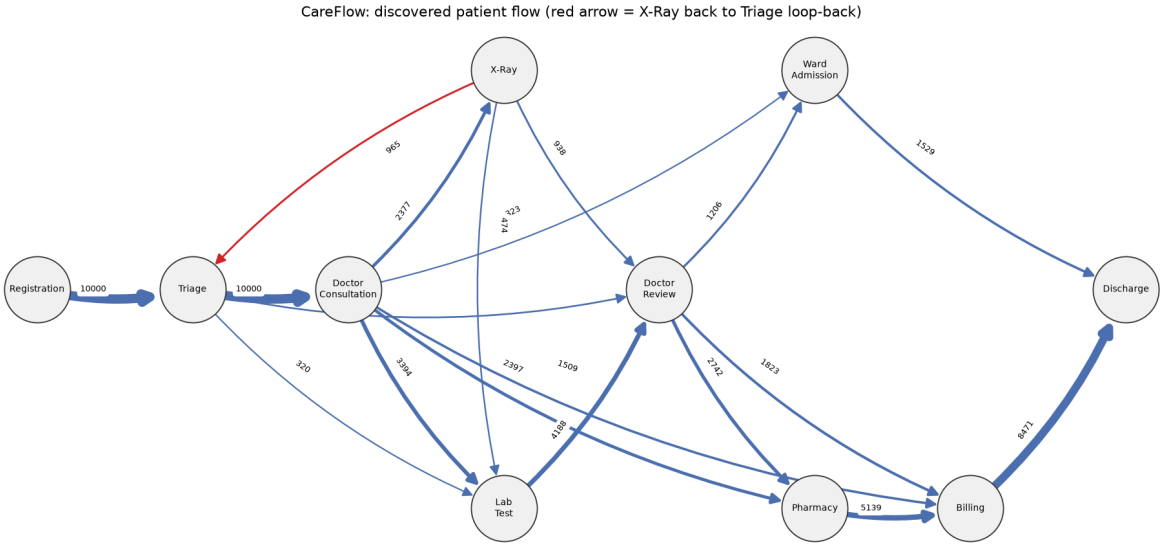


Figure: process map from PM4Py (`docs/process_map.png`).

Waiting time between steps

The dbt `transition_times` model gives the average minutes from one step to the next.

Transition	Patients	Average wait (min)
Lab Test to Doctor Review	4,188	40.0
Doctor Consultation to Pharmacy	2,397	28.4
Triage to Doctor Consultation	10,000	26.6
X-Ray to Triage (loop-back)	965	20.3

Ward Admission to Discharge averages 298.3 minutes across 1,529 patients. This is long by design, since a ward stay takes hours, so it is not treated as a bottleneck.

Planted problems found

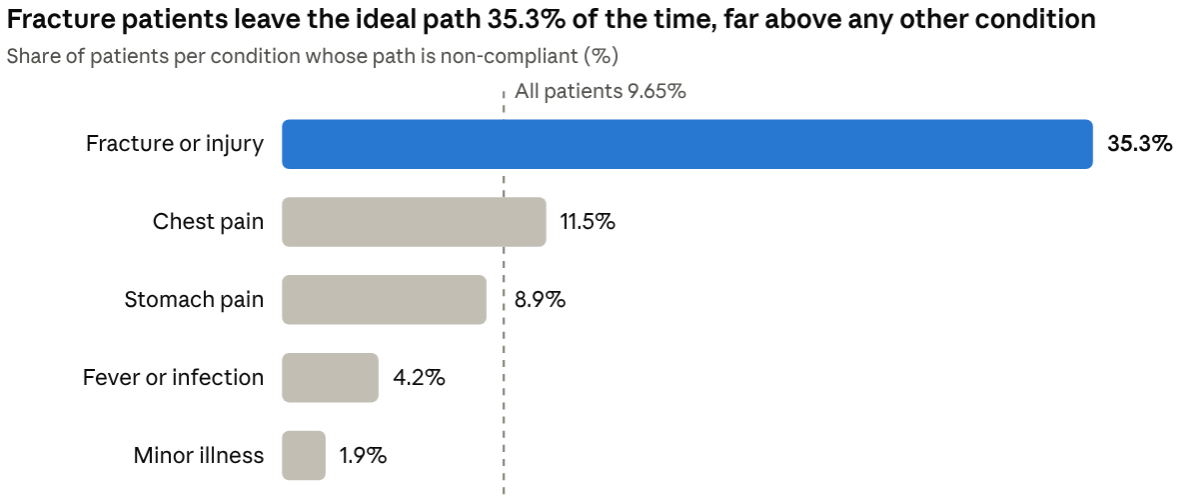
The two timing problems show up clearly when the waits are split by the hour of the day.

Step	Planted hours	Average wait inside (min)	Average wait outside (min)
Doctor Consultation	10:00 to 14:00 and 18:00 to 22:00	48.5	17.8

Step	Planted hours	Average wait inside (min)	Average wait outside (min)
Pharmacy	18:00 to 23:00	44.5	18.2

Conformance

The `case_conformance` model marks a patient as non-compliant if the path goes back to an earlier stage or repeats a step. 9,035 of 10,000 patients (90.35%) are compliant. The 965 who are not are exactly the X-Ray to Triage loop-back patients, which agrees with PM4Py. Fracture or injury patients are affected most, because they need X-Rays most often.



dbt case_conformance model · 10,000 simulated patients

6. Dashboard

The Power BI dashboard lets a manager see the bottlenecks without reading code. It imports the dbt views from BigQuery and has three pages.

Page	What it shows
Page 1	A bar chart of average minutes between steps, with total events (68K) and total patients (10K)
Page 2	The process map
Conformance	Compliant % (90.35%) and Non-compliant % (9.65%) cards, a bar chart of non-compliant share by condition, and slicers for Condition and Age_Group

The slicers filter the conformance visuals, so a manager can ask, for example, how often Senior fracture patients leave the ideal path.

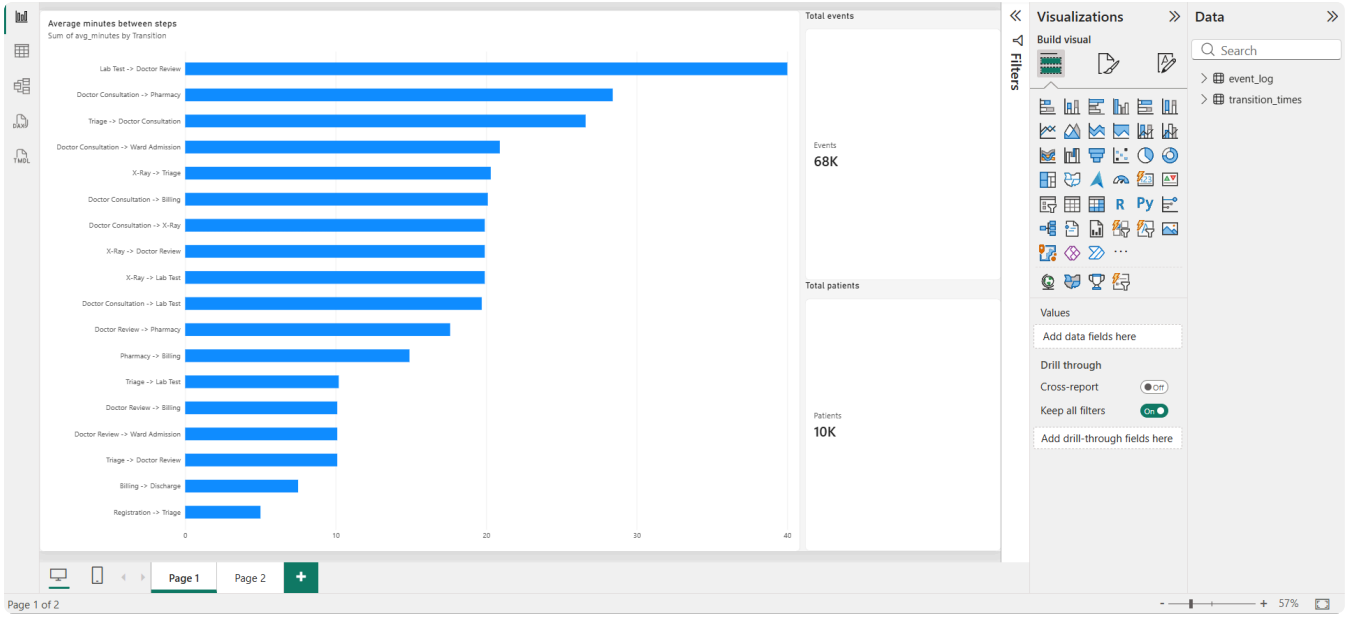


Figure: dashboard page 1 (`docs/dashboard.png`).

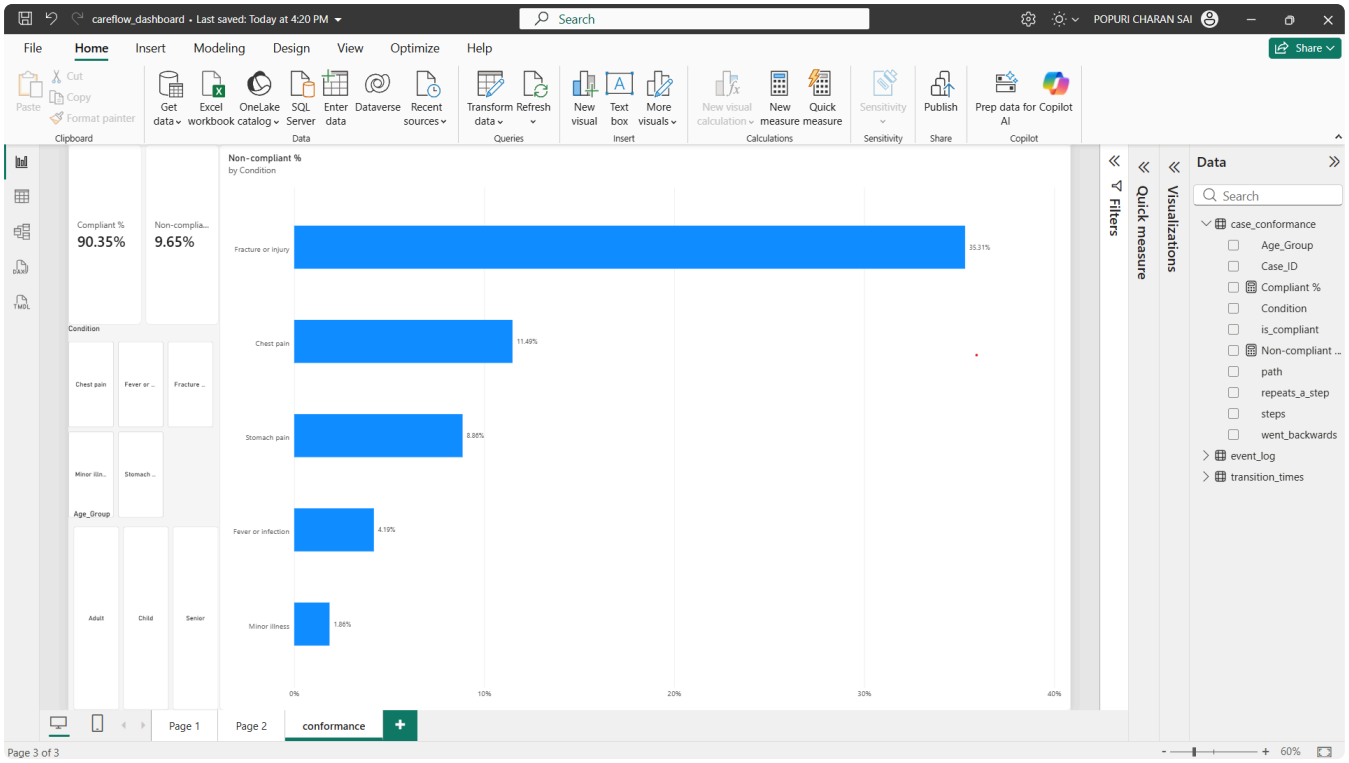


Figure: conformance page (`docs/dashboard_conformance.png`).

7. Challenges and fixes

Most problems came from tool setup rather than from the analysis. Each was fixed and recorded.

Problem	Cause	Fix
BigQuery table load showed an unexpected error	Not confirmed; the console gave only a generic error	Loaded the CSV with a JSON schema file (`bigquery/schema.json`)
dbt failed with a dataset location error	The dataset was in the US region, the data in asia-south1	Created a new dataset `careflow_clean` with location asia-south1 in `profiles.yml`
dbt said no `dbt_project.yml` found	Commands were run from the top-level folder	Run dbt commands from inside `dbt_careflow`
dbt stopped with an SSL error	A network interruption	Re-ran the command; a retry setting is available
dbt `target/` and `logs/` were committed	They were not in `.gitignore`	Removed them with `git rm -r --cached` and added them to `.gitignore`

Problem	Cause	Fix
PM4Py could not draw the map	`matplotlib` was missing from the virtual environment	Installed it and pinned versions in `requirements.txt`
Power BI formula error on a new column	The formula bar already contained `Column =`	Cleared the bar and pasted the formula again

8. Limitations, recommendations and future work

The main limit is that the data is simulated, so the findings reflect how the generator was built and say nothing about a real hospital. Other limits:

- A timestamp marks the start of each step, so waiting time is measured from one start to the next and includes the length of the earlier step.
- The ideal path is a simple rule (move forward, never repeat), not a clinical guideline.
- The Power BI file imports a snapshot of the data and needs a refresh when the BigQuery views change.

If the same patterns appeared in real records, a hospital could act on them in three ways:

1. **Investigate the loop-back.** Find out why 40% of X-Ray patients return to Triage, starting with fracture cases, where 35.3% of patients leave the ideal path.
2. **Staff for busy hours.** Add doctors at 10:00 to 14:00 and 18:00 to 22:00, when the average wait is 48.5 minutes against 17.8 at other times.
3. **Support the evening pharmacy.** Extra staff after 18:00 would target the 44.5-minute average wait.

Future work: run the same pipeline on real, anonymised hospital records, add resource and cost data, and schedule the dbt and PM4Py steps to run automatically.

9. Conclusion

CareFlow delivers a working pipeline from simulated records to a manager's dashboard, and it finds the three planted problems on its own: the 40.6% X-Ray loop-back, the 48.5-minute busy-hour doctor wait and the 44.5-minute evening pharmacy wait. Quality checks at three levels pass, and dbt and PM4Py agree on the 965 non-compliant patients. The method is ready to be tried on real hospital records.

Appendix: repository and how to run

Repository: https://github.com/charansai-popuri/careflow-clinical-pathway-process-mining

Path	Purpose
`data_generator/`	`generate_events.py`, `validate_data.py`, `check_waits.py`
`data/event_log.csv`	The generated event log
`bigquery/schema.json`	BigQuery table definition
`dbt_careflow/`	dbt models `event_log`, `transition_times`, `case_conformance` and their tests
`process_mining/discover_process.py`	PM4Py process discovery from the CSV or BigQuery
`powerbi/careflow_dashboard.pbix`	Power BI dashboard
`docs/`	Design, BigQuery and dbt notes, results, weekly progress, screenshots

To run it, install the packages and run the scripts in this order:

```
pip install -r requirements.txt
python data_generator/generate_events.py
python data_generator/validate_data.py
python data_generator/check_waits.py
python process_mining/discover_process.py
python process_mining/discover_process.py bigquery
```

For the dbt models, run `gcloud auth application-default login` once, create `profiles.yml` in the home `.dbt` folder with the project `careflow-ehr-04`, then run `dbt debug`, `dbt run` and `dbt test` from inside `dbt_careflow`.